

Static vs Dynamic Content



What is the difference between static and dynamic content?
Why do we need HTTP/HTTPS?

Difference Between Static and Dynamic Content

Feature	Static Content	Dynamic Content
Definition	Content that remains the same for every user unless manually changed	Content that changes based on user interaction, database data, time, or conditions
Generated By	Pre-written HTML/CSS files stored on server	Generated at runtime using server-side scripts/programs
Speed	Faster because files are directly served	Slightly slower because processing is required
Database Usage	Usually does not require a database	Often connected to databases
User Interaction	Limited interaction	Highly interactive and personalized
Complexity	Simple to create and host	More complex
Examples	Portfolio website, company information page	Social media feeds, online shopping sites, dashboards
Technologies Used	HTML, CSS, basic JavaScript	PHP, Node.js, Python, Java, databases, APIs

Static Content Example

A simple HTML page:

HTML

```
<h1>Welcome to My Website</h1>
<p>This content is same for every visitor.</p>
```

Every user sees exactly the same page.

Dynamic Content Example

An e-commerce site showing:

- User name after login
- Recommended products
- Cart items

- Live stock updates

Different users see different data because content is generated dynamically.

Why Do We Need HTTP/HTTPS?

HTTP (HyperText Transfer Protocol)

HTTP is a communication protocol used for transferring web pages and data between:

- Client (browser)
- Server (website server)

When you open a website, your browser sends an HTTP request to the server, and the server sends back a response.

Basic Working

1. User enters URL
 2. Browser sends request
 3. Server processes request
 4. Server sends webpage/data
 5. Browser displays it
-

Why HTTP is Needed

Without HTTP:

- Browsers and servers would not know how to communicate
- Web pages could not be transferred over the internet
- Standard rules for requests/responses would not exist

HTTP defines:

- How requests are sent
 - How responses are received
 - Status codes
 - Data formats
 - Headers and methods
-

Common HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Send data
PUT	Update data
DELETE	Remove data

HTTPS (HyperText Transfer Protocol Secure)

HTTPS is the secure version of HTTP.

It uses:

- SSL/TLS encryption

to protect communication between browser and server.

Why HTTPS is Needed

1. Security

Encrypts data so attackers cannot read it.

Example:

- Passwords
- Banking information
- Personal data

2. Authentication

Ensures the website is genuine and not fake.

Example:

A secure certificate proves the site belongs to the real organization.

3. Data Integrity

Prevents data from being modified during transmission.

4. Privacy

Protects user activity from being monitored.

HTTP vs HTTPS

Feature	HTTP	HTTPS
Security	Not secure	Secure
Encryption	No	Yes
Port	80	443
SSL/TLS	Not used	Used
Suitable For	Public/basic information	Login, payments, private data

Real-Life Example

HTTP

Sending a postcard:

- Anyone can read it during delivery.

HTTPS

Sending a sealed envelope:

- Only sender and receiver can read it.

Viva-Style Important Point

Static content is fixed and same for all users, while dynamic content changes according to user interaction or server-side processing.

HTTP enables communication between browser and server, while HTTPS secures that communication using encryption and authentication.



What happens after a URL is entered?
 What is the role of DNS here?
 What is an HTTP response?

What Happens After a URL is Entered?

When you type a URL in a browser and press Enter, several steps happen behind the scenes before the webpage appears.

Example URL:

`https://www.example.com`

Step-by-Step Process

1. Browser Parses the URL

The browser identifies:

- Protocol → `https`
- Domain name → `www.example.com`
- Resource/path → webpage location

The browser now knows:

- Which server to contact
 - Which protocol to use
-

2. DNS Lookup Happens

The browser cannot directly understand domain names like:

`www.example.com`

Computers communicate using IP addresses such as:

`142.250.183.14`

So the browser asks the DNS server:

“What is the IP address of this domain?”

DNS returns the correct IP address.

Role of DNS

DNS = Domain Name System

DNS works like the internet’s phonebook.

It converts:

Human-readable domain names

into

Why DNS is Needed

Without DNS:

- Users would need to remember IP addresses of websites
- Websites could not be easily accessed by names

Instead of typing:

142.250.183.14

we simply type:

google.com

DNS Process

1. Browser checks cache
 2. OS checks cache
 3. Request goes to DNS resolver
 4. Resolver contacts DNS servers
 5. IP address is returned
 6. Browser connects to that IP
-

Viva Point

DNS translates domain names into IP addresses so browsers can locate the correct web server on the internet.

3. Browser Establishes Connection

After getting the IP address:

- Browser connects to the web server
 - For HTTPS, SSL/TLS secure handshake occurs
 - Secure encrypted communication is established
-

4. Browser Sends HTTP Request

The browser sends an HTTP request like:

```
http
GET / HTTP/1.1
Host: www.example.com
```

This asks the server:

“Send me the homepage.”

5. Server Processes Request

The server:

- Receives request
 - Finds requested resource
 - May query databases
 - Generates response
-

6. Server Sends HTTP Response

The server sends data back to browser.

This is called an HTTP response.

What is an HTTP Response?

An HTTP response is the message sent by the server to the client/browser after processing an HTTP request.

It contains:

- Status code
 - Headers
 - Requested data/content
-

Structure of HTTP Response

Example

```
http
HTTP/1.1 200 OK
Content-Type: text/html

<html>
  <body>Hello</body>
</html>
```

Components of HTTP Response

Component	Purpose
Status Line	Shows request result
Headers	Additional information
Body	Actual webpage/data

Common HTTP Status Codes

Code	Meaning
200	Success
404	Page not found
500	Server error
301	Redirect
403	Forbidden

7. Browser Renders the Page

The browser:

- Parses HTML
- Loads CSS, JavaScript, images
- Builds webpage layout
- Displays final webpage to user

Complete Flow Summary

```

User enters URL
↓
Browser parses URL
↓
DNS finds IP address
↓
Browser connects to server
↓
HTTP request sent
↓
Server processes request
↓
HTTP response returned
↓
Browser renders webpage

```

Viva-Style Short Answers

What happens after entering a URL?

The browser performs DNS lookup, establishes connection with the server, sends an HTTP request, receives an HTTP response, and renders the webpage.

What is the role of DNS?

DNS converts domain names into IP addresses so the browser can locate and communicate with the correct web server.

What is an HTTP response?

An HTTP response is the server's reply to a client request containing status code, headers, and requested content such as HTML pages or JSON data.



Can a browser act as a server?

Yes, a browser can act as a server in some situations, but normally a browser acts as a client.

Normal Client-Server Model

Usually:

- Browser → Client
- Web server → Server

Example:

Browser requests webpage
Server sends webpage

Can a Browser Act as a Server?

Yes — under certain conditions.

A browser or a system running the browser can serve data/resources to other devices or applications. This is not the browser's primary role, but it is possible using technologies like:

- Localhost servers

- Peer-to-peer communication
 - Browser APIs
 - Extensions
 - WebRTC
-

Examples

1. Browser Accessing Localhost Server

When developers run:

```
localhost:3000
```

the browser connects to a server running on the same machine.

Here:

- Browser is still client
 - But the same computer also hosts server software
-

2. Browser Sharing Data via WebRTC

Using WebRTC, browsers can directly exchange:

- Files
- Audio
- Video
- Messages

between users without a central server after connection setup.

In this peer-to-peer model, each browser can temporarily behave like both:

- Client
 - Server
-

3. Browser Extensions / Developer Tools

Some browser extensions or tools can expose local services and respond to requests.

4. Browser-Based Hosting Experiments

Some advanced applications use JavaScript and Service Workers to:

- cache content
- respond to requests
- simulate server behavior

Example:

- Progressive Web Apps (PWAs)

Important Clarification

A browser alone is not designed to be a full production web server like:

- Apache HTTP Server
- Nginx
- Node.js

These are dedicated server technologies built to:

- handle many requests
- manage resources
- provide security
- support backend logic

Viva-Style Answer

Normally, a browser acts as a client that requests resources from a web server. However, in some cases such as peer-to-peer communication, localhost environments, or browser APIs like WebRTC, a browser can temporarily perform server-like functions by sharing or responding with data.



Why is Apache popular?
What are modules in Apache?
Can Apache serve dynamic pages?

Why is Apache HTTP Server Popular?

Apache is one of the most widely used web servers because it is:

1. Open Source and Free

- Anyone can use and modify it
- No licensing cost
- Strong community support

This made it highly popular among:

- developers
 - startups
 - educational institutions
-

2. Cross-Platform Support

Apache works on:

- Linux
- Windows
- macOS
- Unix

This flexibility increased adoption.

3. Modular Architecture

Apache supports modules that add extra features like:

- security
- URL rewriting
- authentication
- PHP support

Users can enable only required features.

4. Good Performance and Reliability

Apache is stable and reliable for:

- websites
- enterprise applications

- hosting services

It can handle many client requests efficiently.

5. Strong Community and Documentation

Apache has:

- extensive documentation
- tutorials
- community forums

which makes learning and troubleshooting easier.

6. Supports Dynamic Content

Apache can work with:

- PHP
- Python
- Perl
- CGI
- Java applications

to generate dynamic webpages.

What are Modules in Apache?

Modules are plug-in components that extend Apache's functionality.

Apache follows a modular design, meaning extra features can be added or removed without changing the core server.

Examples of Apache Modules

Module	Purpose
<code>mod_ssl</code>	Enables HTTPS/SSL security
<code>mod_rewrite</code>	URL rewriting and redirection
<code>mod_php</code>	Executes PHP scripts
<code>mod_auth</code>	Authentication and authorization
<code>mod_proxy</code>	Reverse proxy functionality

Why Modules are Useful

Modules make Apache:

- flexible
- customizable
- lightweight

because only needed features are loaded.

Example

If a website uses HTTPS:

Apache loads:

`mod_ssl`

If PHP is needed:

Apache loads:

`mod_php`

Can Apache Serve Dynamic Pages?

Yes.

Apache can serve both:

- Static pages
 - Dynamic pages
-

Static Pages

Apache directly sends files like:

- HTML
- CSS
- Images

Example:

`index.html`

Dynamic Pages

Apache can generate pages dynamically using server-side technologies.

Examples:

- PHP
- Python
- Perl
- CGI scripts

How Dynamic Page Generation Works

Example with PHP

1. Browser requests:

`index.php`

2. Apache forwards PHP code to PHP interpreter
3. PHP executes code
4. Generated HTML is returned to browser

Flow

```

Browser Request
  ↓
Apache Server
  ↓
Server-side Script Executes
  ↓
HTML Generated
  ↓
Response Sent to Browser
  
```

Technologies Apache Uses for Dynamic Content

Apache commonly works with:

- PHP
- Python
- Perl
- CGI
- FastCGI

Viva-Style Short Answers

Why is Apache popular?

Apache is popular because it is free, open source, cross-platform, reliable, modular, and supports both static and dynamic web content.

What are modules in Apache?

Modules are plug-ins that extend Apache functionality, such as SSL support, URL rewriting, authentication, and dynamic scripting support.

Can Apache serve dynamic pages?

Yes. Apache can serve dynamic pages by interacting with server-side technologies like PHP, Python, CGI, and Perl to generate content at runtime.



What is the port for HTTPS?

Why are ports needed?

What is the difference between a port and an IP address?

What is the Port for HTTPS?

The default port number for HTTPS is:

443

When a browser accesses a secure website using HTTPS, communication usually happens through port 443.

Example:

`https://example.com`

Internally means:

`example.com:443`

Common Port Numbers

Protocol	Port Number
HTTP	80
HTTPS	443
FTP	21

Protocol	Port Number
SSH	22
SMTP	25

Why Are Ports Needed?

A computer can run many network services at the same time, such as:

- Web server
- Email service
- Database
- File transfer service

The IP address identifies the computer/device, but ports identify the specific service/application running on that device.

Simple Analogy

IP Address

Like a building address.

Port

Like a room number inside the building.

Example:

Building Address = IP Address
Room Number = Port

The IP gets data to the correct computer, and the port sends it to the correct application.

Example

Suppose a server has IP:

192.168.1.10

Different services may use:

Service	Port
Website	80
Secure Website	443

Service	Port
Database	3306

So requests can reach the correct program.

What is the Difference Between a Port and an IP Address?

Feature	IP Address	Port
Purpose	Identifies device on network	Identifies specific application/service
Scope	Device-level	Application-level
Example	192.168.1.1	80, 443
Used For	Finding the computer	Finding the service/program
Analogy	House address	Door/room number

How They Work Together

A complete network address is:

IP Address + Port

Example:

142.250.183.14:443

Meaning:

- connect to device **142.250.183.14**
- use HTTPS service on port **443**

Viva-Style Answers

What is the port for HTTPS?

HTTPS uses port 443 by default for secure encrypted communication.

Why are ports needed?

Ports are needed to identify specific services or applications running on a device so multiple network services can work simultaneously.

Difference between a port and an IP address?

An IP address identifies a device on a network, while a port identifies a specific application or service running on that device.



so does each webpage have same web-server that browser connects to

Not necessarily. Different webpages can be hosted on:

- the same web server
- different web servers
- multiple servers together

depending on how the website is designed and hosted.

Understanding It Properly

When a browser connects to a website, it connects to the server associated with that website's domain/IP address.

Example:

`https://example.com`

DNS converts the domain into an IP address, and the browser connects to that server.

Case 1: Multiple Webpages on Same Server

A single web server can host many webpages and websites.

Example:

`example.com/home`
`example.com/about`
`example.com/contact`

All may come from the same server.

This is very common.

Case 2: Different Pages from Different Servers

Large websites often distribute content across many servers.

Example in a modern website:

Content	Server Type
Main HTML	Application server
Images	CDN/image server
Videos	Media server
API data	Backend API server

So one webpage may actually load resources from many servers.

Example Flow

When opening a site like:

youtube.com

The browser may connect to:

- one server for HTML
- another for videos
- another for ads
- another for user authentication

Case 3: Load Balancing

Popular websites use multiple identical servers.

The browser connects to one of many available servers chosen by:

- load balancers
- CDN systems
- geographic routing

This improves:

- speed
- reliability
- scalability

Important Concept

Website ≠ Single Physical Server

A website may run on:

- one machine

- many machines
- cloud infrastructure

Browser Perspective

From the browser side:

1. User enters URL
2. DNS resolves domain
3. Browser connects to appropriate server(s)
4. Multiple requests may go to different servers

Viva-Style Answer

No, webpages do not always use the same web server. Small websites may serve all pages from one server, while large modern websites often use multiple servers for HTML, images, videos, APIs, and load balancing.



how the server is identified using dns give an example

How the Server is Identified Using DNS

DNS identifies a server by converting a domain name into its corresponding IP address.

The browser cannot directly understand domain names like:

`google.com`

It needs an IP address such as:

`142.250.183.14`

DNS performs this conversion.

Step-by-Step Example

Suppose you enter:

`https://www.google.com`

Step 1: Browser Extracts Domain Name

The browser identifies:

www.google.com

Step 2: DNS Lookup Starts

The browser asks DNS:

“What is the IP address of **www.google.com**?”

Step 3: DNS Server Responds

DNS returns something like:

142.250.183.14

(This IP may vary by location and time.)

Step 4: Browser Connects to Server

Now the browser knows:

- which machine to contact
- where the website is hosted

The browser sends requests to:

142.250.183.14

usually on:

- port 80 (HTTP)
 - port 443 (HTTPS)
-

Simple Analogy

DNS Works Like a Phone Contact List

Human-Friendly Name	Actual Number
John	9876543210
google.com	142.250.183.14

You remember the name, but the system uses the actual number.

Important Technical Point

DNS mainly maps:

Domain Name → IP Address

The IP address identifies the server/device on the internet.

Real Flow

User enters `google.com`



DNS lookup occurs



DNS returns IP address



Browser connects to that IP



Server responds with webpage

Why This is Necessary

Without DNS:

- users would need to memorize IP addresses
- websites would be difficult to use

Instead of:

`142.250.183.14`

we simply type:

`google.com`

Viva-Style Answer

DNS identifies a server by translating a domain name into its IP address. For example, when a user enters `google.com`, DNS returns the corresponding IP address, allowing the browser to connect to the correct server on the internet.



so the server is that ip address

Yes — in networking, the server is identified using its IP address.

But more accurately:

The IP address identifies the device (machine) on which the server software is running.

Important Distinction

Server

A server is:

- a computer/system
 - or software
- that provides services/resources to clients.

Examples:

- web server
 - database server
 - mail server
-

IP Address

An IP address is:

- the network address of that machine/device

Example:

142.250.183.14

This tells the browser where the server machine exists on the internet.

Real Meaning

When people say:

“Connect to the server”

it practically means:

“Connect to the machine having that IP address.”

Example

Suppose:

google.com → 142.250.183.14

DNS returns the IP.

Then the browser connects to:

142.250.183.14:443

Where:

- 142.250.183.14 → identifies the machine/server
- 443 → identifies HTTPS service

Very Important Concept

A single machine/IP can host:

- multiple websites
- multiple services
- multiple applications

using:

- ports
- virtual hosting

So:

IP address identifies the machine

Port identifies the service

Server software handles requests

Simple Analogy

Networking	Real Life
IP address	Building address
Port	Room number
Server software	Office inside room

Technically Accurate Statement

Instead of saying:

“The IP address is the server”

a better statement is:

“The IP address identifies the machine running the server.”

Viva-Style Answer

The server itself is the machine or software providing services, while the IP address identifies the network location of that server machine so clients can connect to it.



then how a machine becomes a server what additional things it do

A machine becomes a server when it runs software that listens for requests from other computers and provides services/resources to them over a network.

A normal computer and a server computer are fundamentally similar in hardware terms. The difference is mainly in:

- purpose
- software
- configuration
- network accessibility

What Makes a Machine a Server?

1. It Runs Server Software

The machine installs software such as:

- Apache HTTP Server
- Nginx
- Node.js
- Database servers
- Mail servers

This software waits for incoming requests.

Example

If Apache is installed:

Client Browser → Request → Apache Server → Response

Apache listens on:

- port 80 (HTTP)
- port 443 (HTTPS)

and responds with webpages.

2. It Listens Continuously for Requests

Unlike normal applications that run only when opened, server software continuously runs in the background.

This is called:

Listening for connections

Example:

Listening on port 443

meaning:

“I am ready to accept HTTPS requests.”

3. It Has a Public or Reachable IP Address

To act as a server for others:

- the machine must be reachable over a network/internet
- clients must know its IP/domain

Without network accessibility, nobody can connect.

4. It Processes Requests and Sends Responses

A server machine:

- receives requests
- processes them
- returns responses

Example:

- browser requests webpage
 - server sends HTML
-

5. It Can Handle Multiple Clients

Servers are designed to serve many users simultaneously.

Example:

1000 users requesting webpage together

The server manages all those connections.

What Additional Things Does a Server Do?

Function	Description
Listen for requests	Waits for client communication
Process requests	Executes logic/scripts
Serve resources	Sends webpages/files/data
Manage connections	Handles multiple users
Security	Authentication, HTTPS, firewalls
Logging	Stores request/error logs
Database interaction	Retrieves/stores data

Example of Web Server Working

User Opens Website

`https://example.com`

What Happens on Server Machine

1. DNS points to server IP
2. Browser connects to server
3. Apache/Nginx receives request
4. Server processes request
5. Response sent back

Can a Personal Computer Become a Server?

Yes.

Your laptop can become a server if you:

- install server software
- open/listen on ports
- allow network access

Example:

Running a local server:

`localhost:3000`

or hosting your own website.

Simple Analogy

Normal Computer

Like a person using services.

Server

Like a shopkeeper always waiting for customers and serving requests.

Viva-Style Answer

A machine becomes a server when it runs server software that listens for network requests, processes them, and sends responses to clients. The machine must be reachable over a network and continuously provide services such as webpages, databases, or files.



what is application server How is it different from a web server?

Why do enterprise apps need it?

What services does it provide?

What is an Application Server?

An application server is a server that provides an environment to run business logic, backend processing, and enterprise applications.

It sits between:

- the client/browser
- and databases/resources

and handles:

- application execution
 - transactions
 - authentication
 - business rules
 - dynamic processing
-

Simple Understanding

Web Server

Mainly handles:

- HTTP requests
- static content

Application Server

Handles:

- complex application logic
 - dynamic operations
 - backend processing
-

Example

Suppose a user logs into an online banking system.

The application server:

- verifies credentials
- checks database
- calculates balances
- processes transactions
- generates dynamic responses

This is beyond simple webpage serving.

Difference Between Web Server and Application Server

Feature	Web Server	Application Server
Main Purpose	Serve web content	Execute business logic
Handles	Static pages mainly	Dynamic enterprise applications
Protocol Support	Mostly HTTP/HTTPS	HTTP + additional protocols
Business Logic	Minimal	Extensive
Database Interaction	Limited	Heavy interaction
Transaction Management	Usually absent	Supported
Scalability Features	Basic	Advanced enterprise features
Examples	Apache HTTP Server, Nginx	Apache Tomcat, JBoss EAP

Simple Analogy

Component	Real-Life Analogy
Web Server	Reception desk
Application Server	Office staff doing actual work

The web server receives requests.

The application server performs the real processing.

Why Do Enterprise Applications Need an Application Server?

Enterprise applications are:

- large
- complex
- multi-user
- data-intensive

They require features beyond simple webpage delivery.

Enterprise Requirements

1. Business Logic Processing

Example:

- banking calculations
- payroll systems
- inventory management

2. Database Connectivity

Enterprise apps constantly:

- read data
 - update records
 - manage transactions
-

3. Security and Authentication

Application servers provide:

- login management
 - role-based access
 - session handling
-

4. Transaction Management

Example:

Bank transfer must:

- debit one account
- credit another
- rollback if failure occurs

Application servers manage this safely.

5. Scalability

Enterprise apps may handle:

- thousands/millions of users

Application servers support:

- clustering
 - load balancing
 - connection pooling
-

6. Session Management

They track:

- logged-in users
- carts
- workflows

across multiple requests.

What Services Does an Application Server Provide?

1. Business Logic Execution

Runs application code and workflows.

2. Database Services

Provides:

- database connections
 - pooling
 - query management
-

3. Security Services

Handles:

- authentication
 - authorization
 - encryption
-

4. Transaction Management

Ensures reliable operations.

Example:

- online payments
 - banking transfers
-

5. Session Management

Tracks user state during interaction.

6. Messaging Services

Supports communication between distributed systems.

7. Resource Management

Efficiently manages:

- memory
 - threads
 - connections
-

8. Scalability Support

Provides:

- clustering
 - failover
 - load balancing
-

Typical Architecture

```

Browser
↓
Web Server
↓
Application Server
↓
Database
  
```

Example Scenario

Online Shopping Website

Web Server

- receives HTTP request
- serves images/CSS

Application Server

- processes login
- checks inventory
- calculates price
- handles payment

Database

- stores products/users/orders
-

Viva-Style Answers

What is an application server?

An application server is software that provides an environment for running business logic and enterprise applications, handling dynamic processing, transactions, security, and database interaction.

Difference between web server and application server?

A web server mainly serves web content and handles HTTP requests, while an application server executes business logic, manages transactions, security, sessions, and enterprise-level backend processing.

Why do enterprise apps need application servers?

Enterprise applications require application servers because they need complex business processing, scalability, security, transaction management, database connectivity, and support for many simultaneous users.

What services does an application server provide?

Application servers provide business logic execution, database connectivity, transaction management, session handling, security, scalability, messaging, and resource management services.





Why is HTTP stateless?

What is the difference between session and cookie?

How does a server identify a user session?

Why is HTTP Stateless?

HTTP is called stateless because each request is treated as completely independent.

The server does not automatically remember:

- previous requests
- user information
- login state
- earlier interactions

between two HTTP requests.

Example

Suppose a user visits:

`example.com/login`

and then:

`example.com/profile`

By default, HTTP treats these as two unrelated requests.

The server does not inherently know:

“This is the same user.”

Why Was HTTP Designed This Way?

1. Simplicity

Stateless protocols are:

- easier to design
- easier to process

Each request contains all necessary information.

2. Scalability

Because the server does not store client state:

- requests can be handled independently
- load balancing becomes easier
- memory usage is reduced

This allows web servers to support many users efficiently.

3. Reliability

If one request fails, it does not affect others because requests are independent.

Problem with Statelessness

Modern websites need memory of users:

- login sessions
- shopping carts
- preferences

To solve this, technologies like:

- cookies
- sessions
- tokens

are used.

Difference Between Session and Cookie

Feature	Cookie	Session
Stored Where	Client/browser	Server
Contains	Small user data or session ID	User state/data
Security	Less secure	More secure
Size Limit	Small (~4KB)	Larger
Lifetime	Can persist after browser closes	Usually temporary
Accessible By	Client-side	Server-side
Speed	Faster access	Requires server lookup

Cookie

A cookie is small data stored in the browser.

Example:

```
user=12345  
theme=dark
```

The browser automatically sends cookies with future requests.

Session

A session stores user-related information on the server.

Example session data:

```
User ID  
Login status  
Shopping cart  
Permissions
```

The browser usually stores only a session ID.

Important Relationship

Usually:

Cookie stores Session ID

The actual user data remains on the server.

How Does a Server Identify a User Session?

Step-by-Step Process

1. User Logs In

The server creates a unique session ID.

Example:

ABC123XYZ

2. Server Stores Session Data

On the server:

Session ID → User Data

Example:

ABC123XYZ → Sushant's account info

3. Session ID Sent to Browser

The server sends the session ID as a cookie.

Example:

```
http
Set-Cookie: SESSION_ID=ABC123XYZ
```

4. Browser Stores Cookie

The browser saves it automatically.

5. Future Requests Include Cookie

Browser sends:

```
http
Cookie: SESSION_ID=ABC123XYZ
```

6. Server Recognizes User

The server:

- reads session ID
- looks up stored session data
- identifies the user

Now the server knows:

“This request belongs to the logged-in user.”

Complete Flow

```
User logs in
↓
Server creates session
↓
Session ID stored on server
↓
Session ID sent as cookie
↓
Browser sends cookie in future requests
↓
Server identifies user session
```

Simple Analogy

Concept	Real-Life Example
Session	Customer record in store
Session ID Cookie	Token/slip given to customer

The customer shows the token, and the store retrieves their information.

Viva-Style Answers

Why is HTTP stateless?

HTTP is stateless because each request is treated independently, and the server does not automatically remember previous client interactions. This improves simplicity and scalability.

Difference between session and cookie?

A cookie is small data stored in the browser, while a session stores user data on the server. Cookies often store only the session ID used to identify the session.

How does a server identify a user session?

The server creates a unique session ID, stores session data on the server, and sends the session ID to the browser as a cookie. Future requests include the cookie, allowing the server to identify the user session.



What is a servlet?

Which container is commonly used with Tomcat?

What does the container do for a servlet?

What is a Servlet?

A servlet is a Java program that runs on a server and handles client requests to generate dynamic web content.

Servlets are mainly used for:

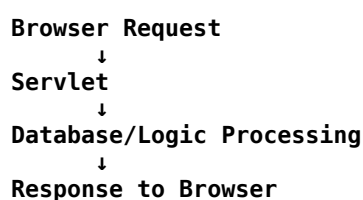
- processing HTTP requests
- interacting with databases
- generating dynamic webpages
- implementing backend logic

They are part of Java web technology.

Simple Definition

A servlet is a server-side Java component that receives requests, processes them, and sends responses back to clients.

Example Flow



Example Use Cases

Servlets can:

- process login forms
- handle registrations
- fetch database records
- manage sessions
- generate HTML dynamically

Basic Servlet Lifecycle

Method	Purpose
<code>init()</code>	Initializes servlet
<code>service()</code>	Handles requests
<code>destroy()</code>	Cleans up before removal

Very Important Method

The `service()` method processes incoming requests.

For HTTP servlets:

- `doGet()` handles GET requests
- `doPost()` handles POST requests

Which Container is Commonly Used with Apache Tomcat?

The commonly used container with Tomcat is:

Servlet Container

also called:

Web Container

Tomcat itself is a servlet container.

What is a Servlet Container?

A servlet container is the environment that:

- runs servlets
- manages servlet lifecycle
- handles communication between web server and servlet

Examples of Servlet Containers

Container	Description
Apache Tomcat	Most popular servlet container
Jetty	Lightweight servlet container
GlassFish	Full Java EE server

What Does the Container Do for a Servlet?

The container provides all runtime support needed for servlets.

Main Responsibilities of Servlet Container

1. Lifecycle Management

The container:

- creates servlet object
- initializes it
- destroys it when needed

Methods managed:

- `init()`
 - `service()`
 - `destroy()`
-

2. Request and Response Handling

The container:

- receives HTTP request
- converts it into Java objects
- passes it to servlet

Example objects:

- `HttpServletRequest`
 - `HttpServletResponse`
-

3. Multithreading

The container handles multiple users simultaneously using threads.

Without this, developers would need to manually manage concurrency.

4. Session Management

Container helps manage:

- user sessions
 - cookies
 - session tracking
-

5. Security Management

Provides:

- authentication
- authorization
- HTTPS support

6. URL Mapping

The container maps URLs to correct servlets.

Example:

`/login` → `LoginServlet`

7. Resource Management

Handles:

- memory
- object pooling
- resource cleanup

Complete Working

```

Browser
  ↓
Tomcat/Servlet Container
  ↓
Servlet Executes
  ↓
Response Returned
  
```

Real-Life Analogy

Component	Analogy
Servlet	Worker
Servlet Container	Manager providing workspace/tools

The servlet focuses on business logic, while the container handles infrastructure tasks.

Viva-Style Answers

What is a servlet?

A servlet is a Java server-side program that processes client requests and generates dynamic responses for web applications.

Which container is commonly used with Tomcat?

Tomcat itself is a servlet container (web container) commonly used to run Java servlets and JSP applications.

What does the container do for a servlet?

The container manages servlet lifecycle, request/response handling, multithreading, session management, security, URL mapping, and resource management for servlets.



What is a WAR file?
Why do we deploy applications?
Can deployment be automated?

What is a WAR File?

A WAR file stands for:

Web Application Archive

It is a packaged file used to deploy Java web applications.

A WAR file contains:

- servlets
- JSP files
- HTML/CSS/JavaScript
- configuration files
- libraries
- application resources

in a single compressed archive.

Purpose of WAR File

It makes Java web applications:

- portable
- easy to distribute
- easy to deploy

WAR File Extension

`myapp.war`

Internally a WAR File Contains

Component	Purpose
WEB-INF/	Configuration and protected files
web.xml	Deployment descriptor
classes/	Compiled Java classes
lib/	JAR libraries
HTML/CSS/JS	Frontend resources

Where is WAR File Used?

WAR files are deployed on:

- Apache Tomcat
- GlassFish
- JBoss EAP

Example

Developer creates:

`shopping-app.war`

Tomcat deploys it and makes it accessible as:

`http://server/shopping-app`

Why Do We Deploy Applications?

Deployment means:

making an application available for users to access and use.

Purpose of Deployment

1. Make Application Accessible

Before deployment:

- application exists only on developer machine

After deployment:

- users can access it through network/internet
-

2. Move Application to Production Environment

Applications are usually developed in:

- local development systems

Deployment places them on:

- production servers
 - cloud servers
 - enterprise infrastructure
-

3. Enable Real Usage

Deployment allows:

- customers
- employees
- clients

to use the application.

4. Centralized Management

Deployed applications can be:

- updated
- monitored

- secured
- scaled

from central servers.

Example

Without deployment:

Only developer can run app

After deployment:

Users worldwide can access it

Can Deployment Be Automated?

Yes.

Modern software development heavily uses automated deployment.

This is called:

CI/CD

Continuous Integration / Continuous Deployment

Why Automate Deployment?

Manual deployment:

- is slow
- error-prone
- repetitive

Automation makes deployment:

- faster
 - reliable
 - consistent
-

Automated Deployment Process

```
Code Push
↓
Build Application
↓
Run Tests
↓
```

Create WAR/JAR
↓
Deploy Automatically

Common Automation Tools

Tool	Purpose
Jenkins	CI/CD automation
GitHub Actions	Workflow automation
Docker	Containerized deployment
Kubernetes	Automated scaling/deployment

Example

Developer updates code in Git repository.

Automation pipeline:

1. compiles application
2. runs tests
3. creates WAR file
4. deploys to Tomcat automatically

No manual copying needed.

Advantages of Automated Deployment

Advantage	Benefit
Faster releases	Quick updates
Reduced errors	Less manual mistakes
Consistency	Same process every time
Easy rollback	Restore previous version
Scalability	Handles large deployments

Viva-Style Answers

What is a WAR file?

A WAR (Web Application Archive) file is a packaged Java web application containing servlets, JSPs, libraries, configuration files, and web resources used for deployment on application servers like Tomcat.

Why do we deploy applications?

Applications are deployed to make them accessible to users in a production environment where they can be centrally managed, scaled, and maintained.

Can deployment be automated?

Yes. Deployment can be automated using CI/CD tools like Jenkins, GitHub Actions, Docker, and Kubernetes to build, test, package, and deploy applications automatically.